

一、Java 特点

- 面向对象（特性：封装、继承、多态、抽象）
- 平台无关性（代码文件：.java，字节码文件：.class）
- 健壮性
- 多线程支持
- 库类丰富

二、数据类型

（一）基本数据类型

- 整数类型：byte、short、int、long
- 浮点类型：float、double
- 字符类型：char
- 布尔类型：boolean

（二）引用数据类型：类、接口、数组、字符串...

三、类与对象

（一）**关系**：类是抽象的，对象是具体的。类定义对象的属性与行为，对象是类的实例。

（二）类：

- 由属性和行为组成
- 构造方法与类名相同，无返回值

四、面向对象

（一）包

- 在 java 文件**首行**使用 **package** 关键字声明包
- 作用：避免命名冲突、代码模块化、访问控制
- 包名小写，使用公司域名逆序；层级以.分隔，对应目录结构

（二）封装

- 优点：安全性、灵活性、易维护性
- 访问控制修饰符：private、protected、public 等

修饰符	类内	包内	子类	其他类
private	✓	✗	✗	✗
无（默认）	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

（三）继承

- 用 **extend** 关键字实现
- Java 只能**单继承**，不能有多个父类
- 子类可以继承父类的**非私有成员**（public、protected）
- 不继承父类的构造方法，但可以通过 **super()** 调用，super() 必须放在构造方法**第一行**

（四）多态

- 两种实现：编译时多态（静态多态）-方法重载（Overloading）、运行时多态（动态多态）-方法重载（Override）+父类引用指向子类对象
- 方法重载（Overloading）：**同一个类**中，定义多个**同名不同参**的方法
- 方法重写（Override）：重定义父类方法，**方法名、参数列表、返回类型**必须与父类一致；访问权限**不更严格**于父类

（五）抽象类

- 目的：代码复用
- 用 **abstract** 关键字声明类和方法
- **不能被实例化**
- 非抽象类子类必须实现**所有抽象方法**
- 抽象类可以包含字段、构造方法、具体方法、抽象方法

（六） 接口

- 目的：定义规范
- 用 **interface** 关键字声明接口，用 **implements** 关键字实现
- 允许多**继承**，一个类可以实现多接口
- 使用接口的类必须实现所有抽象方法
- 接口方法默认是 public abstract，字段默认是 public static final
- 不包含构造方法

五、 异常处理（代码详见 exceptionHandle）

（一） 异常定义

- 检查型异常：必须显示处理（try-catch 语句，签名方法加上 throws）
- 非检查型异常
- 错误

（二） 异常处理语法

- try-catch-finally
- throws 声明异常：把异常给调用者处理
- throw 手动抛出异常

（三） 自定义异常

- 继承 **Exception**（检查型）或 **RuntimeException**（非检查型）
- 构造方法需含 super(...)

六、 集合框架

（一） 顶层接口

- **Collection**：单列数据的根接口
- **List**：**有序、可重复**（如 ArrayList、LinkedList）
- **Set**：**无序、不可重复**（如 HashSet、TreeSet）
- **Map**：双列数据（键值对），**键不可重复**（如 HashMap、TreeMap）

（二） List 接口

1. ArrayList

- **查询快、增删慢**
- add() 添加元素；get() 获取元素；remove() 删除元素；size() 获取长度

2. LinkedList

- **增删快、查询慢**

3. 遍历方式

- for (int i = 0; i < list.size(); i++) {}
- for (DataType item: list) {}
- Iterator<DataType> iterator = list.iterator();
While (iterator.hasNext()) {
 Iterator.next();
}

（三） Set 接口

1. HashSet

- 无序、唯一、允许 null 值
- add() 添加元素

2. TreeSet

- 自然排序或自定义排序
- 元素必须实现 Comparable 接口或传入 Comparator

(四) Map 接口

3. HashMap

- 键唯一、无序、允许 null 键和 null 值

4. TreeMap

- 键自然排序或自定义排序

(五) 集合工具类 (Collections)

- sort(): 排序
- remove(): 反转
- max(): 最大值

七、泛型 (代码详见 generic)

(一) 作用: 类型安全、代码复用

(二) 泛型类与泛型方法的定义语法: 类名后添加 <?>, 字段添加 ? 类型属性

(三) 使用方法: 使用 ? 类型字段/方法

(四) 类型通配符与边界

- <?>: 未知类型 (任意类型)
- <? extend Class>: 上限通配符 (接受 Class 及其子类)
- <? super Class>: 下限通配符 (接受 Class 及其父类)

八、枚举 (代码详见 enumClass)

(一) 优势: 类型安全、可拓展性

(二) 用法:

- 添加字段与方法
- 实现接口

九、流

(一) 类别

- 按数据单位分: 字节流、字符流
- 按方向分: 输入流、输出流
- 按功能分: 节点流、处理流

(二) 文件操作 (代码详见 stream.fileOperation)

- 文件复制: 使用 FileInputStream()、FileOutputStream() 方法
- 文件读取: 使用 FileReader() 方法
- 缓冲流优势: 通过内存缓冲区减少直接读写磁盘的次数, 提升 I/O 效率

(三) 标准输入输出流 (代码详见 stream.systemStream)

1. 标准流作用

- System.in: 标准输入流, 类型为 InputStream, 用于从控制台读取用户输入
- System.out: 标准输出流, 类型为 PrintStream, 用于向控制台打印信息
- System.err: 标准错误流, 通常用于输出错误信息

2. System.in: 只能读取单字节

3. Scanner 类

4. System.out

- `print()`: 输出不换行
- `println()`: 输出并换行
- `printf()`: 格式化输出
- `setOut()`: 输出重定向

(四) 对象序列化与反序列化 (代码详见 `stream.serialization`)

1. 作用: 序列化将对象转换为字节流, 方便存储或网络传输; 反序列化将字节流恢复为对象

2. 序列化实现

- 对象应实现 `Serializable` 接口
- 对象中用 `transient` 关键字修饰的字段/方法不参与序列化
- 调用 `objectOutputStream()` 方法

3. 反序列化实现

- 调用 `objectInputStream()` 方法

(五) NIO

1. 优势: 简化文件操作

2. 核心组件

- 通道 (`Channel`)
- 缓冲区 (`Buffer`)
- 选择器 (`Selector`)

十、多线程编程

(一) 进程与线程

1. 概念:

- 进程: 一个正在执行的程序实例, 拥有独立的内存空间
- 线程: 进程内的执行单元, 共享进程资源。一个进程内可以有多个并发线程

2. 不同点:

	资源开销	内存空间	通信	系统开销
进程	高	独立	复杂	高
线程	低	共享	简单	低

3. 意义: 性能优化与效率提升、用户体验增强、高并发处理能力、资源利用效率最大化、异步编程与模块化设计

4. 线程生命周期:

- 新建 (`new`): 线程创建但未启用
- 就绪 (`Runnable`): 线程已启动, 等待 CPU 分配时间片
- 运行 (`Running`): 线程正在执行代码
- 阻塞 (`Blocked`): 线程因等待资源 (如锁、I/O) 暂停执行
- 终止 (`Terminated`): 线程执行完毕或异常终止

(二) 线程的创建与启动 (代码详见 `multithread.thread`)

- 继承 `Thread` 类: 代码简单, 但受单继承限制, 可拓展性差
- 实现 `Runnable` 接口: 需通过 `Thread` 包装, 可拓展性好

(三) 线程同步与锁机制 (代码详见 `multithread.synchronizedAndLock`)

1. 线程安全问题: 多个线程同时访问共享资源时, 由于操作的非原子性等原因, 可能导致数据不一致或程序行为异常

2. 同步代码块: 通过 **synchronized** 关键字锁定对象或类, 确保同一时间只有一个线程执行同步代码

3. 同步方法: 在方法声明时使用 **synchronized** 关键字

4. 显示锁: 通过 `java.util.concurrent.locks.Lock` 接口的实现类

(四) 线程间通信 (代码详见 `multithread.threadCommunication`)

1. Java 为 **Object** 类提供了基础线程间通信手段

- `wait()`: 使当前线程进入等待状态并释放对象锁, 直到其他线程调用同一对象的 `notify()` 或 `notifyAll()` 方法唤醒
- `notify()`: 唤醒一个正在等待该对象锁的线程
- `notifyAll()`: 唤醒所有正在等待该对象锁的线程
- 以上方法必须在同步代码块或同步方法中调用

十一、网络编程

(一) 网络编程基础

1. 核心概念

1.1. IP 地址

- 作用: 唯一标识网络中的设备, 分为 IPv4 和 IPv6
- IPv4 约 2^{32} 个地址, IPv6 约 2^{128} 个地址
- 特殊地址: 127.0.0.1 (本地回环测试), 0.0.0.0 (所有接口), 255.255.255 (广播), 10.0.0.0/8、176.16.0.0/12、192.168.0.0/16 (私有地址)

1.2. 端口

- 作用: 标识设备上的应用程序, 范围 $0 \sim 65535$ (2^{16} 个)
- 分类: 知名端口 ($0 \sim 1023$), 注册端口 ($1024 \sim 49151$), 动态端口 ($49152 \sim 65535$)

2. 网络协议与分层模型

2.1. TCP (传输控制协议)

- 特点: 面向连接、可靠传输 (确认重传机制)、有序交付
- 适用场景: 文件传输、网页加载、电子邮件

2.2. UDP (用户数据报协议)

- 特点: 无连接、不可靠 (可能丢包)、高效
- 适用场景: 实时音视频、在线游戏、DNS 查询

2.3 TCP/IP 参考模型

- 应用层: 用户接口、格式转换、会话管理
- 传输层: 端到端通信
- 网络层: 路由和寻址
- 网络接口层: 传输原始比特流 (电缆、光纤)

2.4 分层实现示例: http 客户端

- 应用层: 构造 http 请求报文 (GET/index.html HTTP/1.1)
- 传输层: 通过 TCP 建立连接 (Socket)
- 网络层: 操作系统处理 IP 寻址和路由
- 网络接口层: 网卡驱动程序将数据转换为比特流

(二) TCP Socket 编程

1. Socket

- 网络编程中的基本单元, 用于描述网络上的一个进程
- 包含 IP 地址和端口号, 用于唯一标识网络上的一个进程
- 可以通过 Socket 进行网络通信

2. Socket 编程步骤

2.1 创建 Socket

- 客户端（线程不安全）：`Socket socket = new Socket("server_ip", port);`
- 服务端（线程安全）：`ServerSocket serverSocket = new ServerSocket(port);`

2.2 建立连接

- 客户端：通过 `connect()` 方法连接客户端
- 服务端：通过 `accept()` 方法等待客户端连接
- 建立连接后，双方可以将通过输入输出流进行数据交换
- 通信结束后，关闭连接

3. 服务端/客户端实现（代码详见 `network.tcp`）

4. 关键代码解析

4.1. `ServerSocket.accept()`：阻塞方法，知道客户端连接

4.2. 多线程处理：每个客户端连接独立线程，避免阻塞主线程

4.3. 输入输出流：

- `socket.getInputStream()`：获取输入流（读取客户端数据）
- `socket.getOutputStream()`：获取输出流（向客户端发送数据）

（三）UDP 编程

1. UDP

- 无连接：无需预先建立连接，直接发送数据包
- 数据包边界：每个 `DatagramPacket` 独立处理，不保证顺序
- 适用场景：实时性要求高（如视频会议），允许少量数据丢失（如传感器数据上传）

2. 服务端/客户端实现（代码详见 `network.udp`）

（四）TCP 与 UDP 的差异

对比维度	TCP	UDP
连接方式	面向连接（三次握手）	无连接
可靠性	可靠（确认重传机制）	不可靠（可能丢包）
传输效率	较低（需建立连接、保证顺序）	较高（无连接开销）
适用场景	文件传输、网页浏览	实时音视频、在线游戏

十二、数据库编程（代码详见 `database`）

（一）JDBC 概述

1. JDBC

1.1. 定义：Java 提供的数据库统一访问接口

1.2. 价值：数据库无关、简化开发

2. 架构

2.1. 四层架构

- 应用程序：
- JDBC API：
- JDBC 驱动管理器：
- 数据库驱动：

3. JDBC API-应用程序接口层

3.1. 定位：Java 应用程序与数据库之间的编程接口

3.2. 核心包：`java.sql`、`javax.sql`

3.3. 主要功能：

- 建立数据库连接
 - 执行 SQL 语句
 - 处理结果集
 - 管理事务
4. JDBC 驱动管理器-中央调度器
 - 4.1. 定位：JDBC 架构中的交通指挥中心
 - 4.2. 核心类：DriverManager
 - 4.3. 主要功能：
 - 加载并注册数据库驱动
 - 建立数据库连接
 - 管理多个驱动实例
 5. 数据库驱动-厂商实现层
 - 5.1. 定位：应用程序和数据库管理系统之间的适配器
 - 5.2. 核心接口：java.sql.Driver
 - 5.3. 主流数据库驱动：
 - MySQL：com.mysql.cj.jdbc.Driver
 - Oracle：oracle.jdbc.OracleDriver
 - PostgreSQL：org.postgresql.Driver
 - SQL Server：com.microsoft.sqlserver.jdbc.SQLServerDriver

（二）连接数据库

1. 加载数据库驱动
 - 传统方式（JDBC4.0 前）：`Class.forName("org.postgresql.Driver");` //显示加载驱动类
 - 现代方式（JDBC4.0+）：自动通过 SPI 加载驱动，无需显示加载驱动类
2. 建立数据库连接
 - 2.1. 核心接口：Connect（代表一个物理数据库连接）
 - 2.2. JDBC URL：数据库连接等待定位标识，有定位功能、声明协议、配置载体三大作用
 - 2.3. URL 标准格式解析：`jdbc:<子协议>://<主机>:<端口>/<数据库名>?<参数键值对>`
 - 2.4. Connect 对象：数据库会话的核心
 - 2.5. 获取连接

（三）执行 SQL 语句

1. Statement（普通语句）
 - 1.1. 定义：JDBC 中最基本的接口，用于执行静态的 SQL 语句（即不包含参数的 SQL 语句）。通常用于执行简单的查询或更新操作
 - 1.2. 特点：
 - 用于执行静态 SQL 语句
 - 每次执行 SQL 语句时，都需要把完整的 SQL 语句作为字符串传递
 - 容易受到 SQL 注入攻击
 - 1.3. 常用方法：
 - `executeQuery(String sql)`：执行 SELECT 查询，返回一个 ResultSet 对象
 - `executeUpdate(String sql)`：执行 INSERT、UPDATE 或 DELETE 操作，返回受影响的行数
 - `execute(String sql)`：执行任意 SQL 语句，返回一个布尔值，标识是否返回了 ResultSet
2. PreparedStatement（预编译语句）

2.1. 定义: Statement 的子接口, 用于执行预编译的 SQL 语句。支持参数化查询, 能够有效防止 SQL 注入, 并且性能更高

2.2. 特点:

- 预编译 SQL 模板, 高效执行多次
- 通过占位符? 传递参数, 防止 SQL 注入

2.3. 常用方法:

- setXxx(int parameterIndex, Xxx value): 设置参数值 (如 setString、setInt 等)
- executeQuery(): 执行 SELECT 查询, 返回一个 ResultSet 对象
- executeUpdate(): 执行 INSERT、UPDATE 或 DELETE 操作, 返回受影响的行数
- Execute(): 执行任意 SQL 语句, 返回一个布尔值, 标识是否返回了 ResultSet

(四) 处理结果集

1. 遍历结果集

- next(): 移动游标到下一条
- getInt(): 获取 int 值, 参数可为列名/列索引 (索引从 1 开始)
- getString(): 获取 String 值, 参数可为列名/列索引 (索引从 1 开始)